

Trekio - Aplicação de Trilhos e Caminhadas

João Pedro Grave Martins
Tiago de Bragança Praia
Afonso Duarte de Freitas e Sá Lopes

Orientadores: Paulo Alexandre Leal Barros Pereira
Pedro Alexandre de Seia e Cunha Ribeiro Pereira

Relatório do projeto realizado no âmbito de Projecto e Seminário
Licenciatura em Engenharia Informática e de Computadores

Junho de 2026

INSTITUTO SUPERIOR DE ENGENHARIA DE LISBOA

Trekio - Aplicação de Trilhos e Caminhadas

50055 João Pedro Grave Martins

51497 Tiago de Bragança Praia

51498 Afonso Duarte de Freitas e Sá Lopes

Orientadores: Paulo Alexandre Leal Barros Pereira

Pedro Alexandre de Seia e Cunha Ribeiro Pereira

Relatório do projeto realizado no âmbito de Projecto e Seminário
Licenciatura em Engenharia Informática e de Computadores

Junho de 2026

Resumo

Os caminhantes enfrentam um problema geral ao planejar as suas atividades: encontrar um ou mais trilhos adequados aos seus interesses. Esta é a principal motivação para o desenvolvimento deste projeto. O objetivo deste projeto é criar uma aplicação, Trekio, focada na acumulação de informação sobre possíveis trilhos, que os utilizadores possam ter interesse em percorrer, ou ganharem o conhecimento da existência deles. Foi então projetado como componentes principais um servidor, onde são guardadas todas as informações sobre os utilizadores, trilhos e caminhadas exercidas pelos utilizadores, e uma aplicação mobile para Android. Também foi projetado a possibilidade de execução em Web, não tendo possibilidade de executar em iOS por falta de dispositivos deste sistema.

Utilizando um Modelo de Linguagem de Grande Escala (LLM), a aplicação fornecerá, em tempo real, várias sugestões de trilhos ao utilizador com base no seu pedido (através de uma interface de *chatbot* ou de parâmetros definidos). Os utilizadores poderão conversar com um *chatbot* para obter assistência dentro da aplicação. O sistema também suporta a importação de trilhos de outras aplicações. Com base nos trilhos anteriormente realizados pelo utilizador, poderão ser sugeridas novas rotas semelhantes ou rotas já efetuadas, ao mesmo tempo que será disponibilizado um histórico para consulta futura.

Palavras-chave: Trekio; .

Utilização de Inteligência Artificial

Neste trabalho foram utilizadas ferramentas de inteligência artificial generativa como apoio no desenvolvimento de código e na necessidade de resolução de problemas. Foram utilizadas ferramentas como Claude, Gemini e GitHub Copilot, sempre com o objetivo em otimizar a criação de código e fazer análises de documentação mais complexa sobre algumas bibliotecas utilizadas. Alguns exemplos são as bibliotecas Mapbox Android e Mapbox GL JS, utilizadas para instanciar os mapas em Android e Web.

Índice

1	Introdução	1
1.1	Enquadramento	1
1.2	Estado da arte	1
1.3	Objetivos e metas	2
1.4	Resumo da solução proposta	2
2	Formulação do problema	3
2.1	Tecnologias adotadas	3
3	Software	5
3.1	Frontend	5
3.1.1	Tecnologia para Alvos	5
3.1.2	Biblioteca de Mapa	5
3.2	Backend	6
3.2.1	Segurança	6
3.2.2	Tratamento de Erros	7
3.2.3	Acesso a Dados	7
3.2.4	Comunicação em Tempo Real	7
3.2.5	Cálculo de distância de Percurso	8
3.3	Ktor	8
4	Arquitetura	11
4.1	Frontend	11
4.1.1	Biblioteca KMP para Mapa	11
4.2	Backend	12
4.2.1	Modelo de Dados	12
4.2.2	Modelo ER	13
4.2.3	Rotas	15
5	Conclusões	19

Lista de Figuras

4.1	UML do Modelo de Dados desenvolvido	12
4.2	UML do Modelo ER desenvolvido	14

Capítulo 1

Introdução

Este capítulo tenciona explicar ao leitor o problema encontrado, a importância deste, o que já existe como solução para este, qual será a solução e como está organizado o relatório.

1.1 Enquadramento

Durante os tempos modernos, os caminhantes e/ou corredores têm um problema principal relativamente aos trilhos/caminhos que pretendem percorrer: encontrar um que cumpra todas as suas preferências e expectativas.

Este paradigma cria diversas dificuldades e obstáculos, como perda de tempo em procura, planeamento pessoal excessivo, não registo de informações de estatística da caminhada/corrida, entre outros.

Visando este cenário, considerou-se pertinente o desenvolvimento de uma aplicação que resolva estes problemas e outros encontrados ao longo do seu desenvolvimento.

1.2 Estado da arte

Relativamente ao mercado desta área, existem diversas soluções nas diferentes plataformas de aplicações com funcionalidades notáveis e extensas.

Apesar disto, considera-se que o desenvolvimento de uma aplicação deste tipo pode ser algo importante devido à exploração e estudo de novas ferramentas da área onde esta se encontra inserida.

1.3 Objetivos e metas

Este projeto visa como principal objetivo o desenvolvimento de uma aplicação com extensão a multi-plataforma para caminhantes e corredores que disponibilize funcionalidades para a criação e caminhada de trilhos, guardando diversas estatísticas sobre estes, como distância total percorrida, número total de trilhos percorridos, entre outras.

Como objetivos mais específicos, tem-se então:

- Integração de um mapa interagível pelos utilizadores;
- Autenticação via OAuth/OpenID;
- Integração de um *chatbot* através de um LLM.

1.4 Resumo da solução proposta

Como solução ao problema descrito, desenvolveu-se então uma aplicação, principalmente para dispositivos Android, com KMP, a biblioteca MapBox e diversas outras tecnologias e ferramentas descritas ao longo deste relatório.

Esta aplicação visa ter como funcionalidades as seguintes:

- Criação e autenticação de utilizadores através de plataforma Google ou email e password;
- Criação e caminhar um trilho desenvolvido pela comunidade ou pelo próprio utilizador;
- Interação com um *chatbot* para ajuda na aplicação;
- Acesso a dados e estatísticas do utilizador.

Capítulo 2

Formulação do problema

2.1 Tecnologias adotadas

Foram escolhidas as seguintes tecnologias para o desenvolvimento desta aplicação:

- KMP para o desenvolvimento da aplicação em si, permitindo esta ser multi-plataforma;
- Exposed para acesso à base de dados;
- Docker para isolamento e *load balancing*;
- MapBox como biblioteca de implementação de mapas;
- Gradle para compilação e gestão de dependências

Capítulo 3

Software

3.1 Frontend

3.1.1 Tecnologia para Alvos

Ao iniciar o projeto estávamos em dúvida se deveríamos apenas trabalhar sobre o alvo Android ou se também faria sentido trabalhar na Web. Isto obrigou-nos a fazer uma escolha, utilizar Kotlin Multiplatform ou utilizar *Android Studio* e *React*.

Tendo em conta a nossa incerteza sobre os alvos e a natureza do nosso projeto, decidimos prosseguir com KMP, já que também existe o objetivo de termos a possibilidade de deixar disponível um fácil transição para iOS.

Esta decisão acabou por levar a algumas consequências negativas no decorrer do projeto, devido há necessidade de utilizar bibliotecas KMP, que nem sempre existem por ainda ter uma comunidade pequena.

3.1.2 Biblioteca de Mapa

Tínhamos uma decisão importante neste componente. Qual biblioteca de mapas é que poderíamos utilizar, sendo preferencialmente uma biblioteca KMP.

Começamos por avaliar 2 bibliotecas, *OpenStreetMap* e *Google Maps API*, mas descartamos por não serem KMP e necessitarem de informações bancárias.

Após uma pesquisa mais profunda, descobrimos 2 bibliotecas, *MapLibre* e *MapBox*. O *MapLibre* tinha mapa KMP e não necessitava de nenhum tipo de autenticação, enquanto que o *MapBox* apenas tem bibliotecas separadas, *MapBox Android* e *MapBox GL JS*, necessita de autenticação e para emails que não sejam universitários é necessário colocar dados de conta bancária. Importante denotar que *MapLibre* é uma bifurcação de *MapBox*.

Tendo isto em conta, decidimos começar a nossa implementação a partir do *MapLibre*, mas ao decorrer do projeto fomos descobrindo falhas no código original da biblioteca e descobrimos também que a biblioteca não tinha uma implementação completa para a *Web*. Ficamos então com outra decisão para tomar, continuar com o *MapLibre*, resolver os problemas encontrados e utilizar essa resolução como indício de contribuição para a comunidade KMP ou transitar

para *MapBox*, ficando com a necessidade de trabalhar com os *expect/actual* para cada alvo.

Sobre esta decisão, decidimos optar pelo mais seguro, tendo em conta o tempo desperdiçado nos problemas do *MapLibre*, escolhendo transitar para o *MapBox*.

3.2 Backend

3.2.1 Segurança

Apesar de não trabalharmos com dados sensíveis dos clientes, como por exemplo pagamentos, ainda é necessário ter atenção à segurança dos dados, já que, muitas pessoas utilizam sempre a mesma palavra-passe para o email e é necessário minimizar os riscos de fuga de informação.

Autenticação

É importante que o utilizador possa comprovar que realmente é o dono da conta e nesse sentido temos 2 métodos de autenticação. O utilizador pode escolher autenticar-se por *OAuth* da *Google*, ou por “email” e “password”. Após isso usamos um método de tokens de “access” e “refresh”. O token de acesso é um JWT que guarda o nome do utilizador e tem uma vida útil de 15 minutos. O “refresh” token é o token de longa duração utilizado para recriar o token de acesso e é utilizado apenas quando o token de acesso expira, ou em métodos importantes como eliminação e alteração de dados da conta. O tempo de vida útil é de 90 dias, mas como manter um token durante tanto tempo pode ser arriscado, decidimos que o “refresh” token só pode ser utilizado uma vez, recriando sempre um novo. Este token é gerado utilizando o método *SecureRandom.getInstanceStrong()* da biblioteca “java.util” e tem tamanho de 32 bytes. Ambos são transmitidos ao servidor a partir do cabeçalho *Authorization*, passado nos pedidos HTTP, sempre com o esquema *Bearer*.

```
ByteArray(TOKEN_BYTES).let { byteArray ->
    SecureRandom.getInstanceStrong().nextBytes(byteArray)
    getURLencoder().encodeToString(byteArray)
}
```

Encriptação

A encriptação é tão importante quanto a autenticação, pois caso alguém tenha acesso aos dados da nossa base de dados, caso os valores importantes estejam guardados sem encriptação, o invasor terá acesso a todas as contas do sistema.

Para suprimir esta necessidade utilizamos 2 métodos de encriptação, BCrypt e SHA256. O motivo para o qual usamos 2 métodos e não apenas 1 tem haver com a necessidade de encriptação, já que utilizamos o BCrypt para encriptar as palavras-chave e o SHA256 para encriptar os tokens. Mesmo que o token já tenha uma forte encriptação devido ao *SecureRan-*

dom, decidimos que colocar o valor diretamente na base de dados ainda era acompanhado de riscos, logo decidimos utilizar este método de encriptação.

Por outro lado, a palavra-chave sendo uma informação mais importante, pelos motivos indicados na secção 3.2.1, deve ter um método de encriptação mais forte e por esse motivo escolhemos o BCrypt, vindo da biblioteca de segurança do sistema *Sping*, (*springframework.security*).

3.2.2 Tratamento de Erros

Para o tratamento de erros, tomamos uma abordagem de unir os tipos de erros dentro de *sealed class* para cada serviço disponível, no caso User, Trail e Hike, sendo estas *sealed classes* extensões de uma outra *sealed class* geral chamada “DomainError”, que contém possíveis erros que possam acontecer ainda no tratamento dos pedidos HTTP.

Para encapsular os erros utilizamos uma *sealed class* “Either”, tendo o método “failure” e o método “success”, sendo “failure” do tipo uma das classes de erro e “success” o resultado desejado ao pedido exercido. Esta classe é uma imitação da classe *Result* do *Kotlin*, mas como só tivemos conhecimento da sua existência já numa fase mais avançada do projeto, decidimos seguir com a classe que já tínhamos trabalhado.

Comparativamente a outras possíveis implementações, como lançar exceções, a nossa decisão pode apresentar menor overhead, evitando os custos associados ao lançamento de exceções.

Este *overhead* advém da necessidade de criação de um stack trace, coisa que não acontece com a classe “Either”. Outras vantagens são a nível computacional, como melhor previsibilidade dos erros e menor necessidade de compreensão das possíveis exceções lançadas pelo método utilizado.

3.2.3 Acesso a Dados

A tecnologia que escolhemos para a camada de acesso a dados foi o *Exposed*. Tínhamos de decidir entre escolher tecnologias como *JDBC* ou *JDBI* e *Exposed* e a nossa decisão foi sem sobra de duvidas *Exposed*.

O motivo mais importantes desta escolha é a API fortemente tipificada que fornece verificação de erros em tempo de compilação, ao contrário das outras opções, que só demonstram os erros em tempo de execução. Outros motivos são a vantagem de podermos usar a DSL ao invés de escrever as *queries* em formato de *string*.

3.2.4 Comunicação em Tempo Real

Este é uma componente importante do nosso projeto, pois foi projetado que os utilizadores possam percorrer caminhos observando a posição de outros utilizadores no mesmo percurso. Para podermos resolver esta questão, é necessário que os utilizadores disponibilizem as suas

localizações ao nosso servidor. Tendo em conta que estamos a utilizar localização exata do utilizador, e esta localização é atualizada num espaço muito curto, decidimos utilizar *WebSockets*.

Os *WebSockets* são mais vantajosos que os *SSE*, já que são *full-duplex* bidirecionais enquanto os *SSE*, são *simplex* unidirecionais. Como a localização será enviada de forma rápida, avaliamos que manter a ligação aberta com *WebSocket* é menos custoso do que fazer um pedido HTTP a cada nova localização.

Para apoiar os *WebSockets*, projetamos utilizar *Redis Pub/Sub* com o objetivo de ter uma comunicação dos *WebSockets* entre servidores. Esta implementação ainda está em desenvolvimento e terá importância quando for desenvolvido o balanceamento de carga a partir do *nginx*.

3.2.5 Cálculo de distância de Percurso

Tendo em conta que precisamos saber a distância de um percurso, tivemos de escolher um método matemático que pudesse ser usado para fazer este cálculo.

Tínhamos então 2 opções, distância euclidiana e distância de haversine e decidimos escolher a distância de haversine. Isto porque a distância euclidiana calcula a distância em linha reta entre dois pontos, o que para pontos distantes poderia causar problemas, enquanto que a distância de haversine leva em conta a curvatura da Terra.

Obviamente para distância curtas, ambos os métodos funcionam bem, levando vantagem a distância euclidiana, mas como é possível haverem pontos de um percurso com distância largas, decidimos optar pelo método mais preciso, mesmo perdendo velocidade de computação.

Vale ressaltar que a distância de haversine normalmente é o método utilizado para calcular distância com coordenadas GPS, sendo utilizado pelo *Google Maps* em certos casos.

3.3 Ktor

A adoção do *Ktor* para o desenvolvimento do servidor (*backend*) surgiu como uma consequência natural e estratégica da escolha do KMP como tecnologia central do projeto. Inicialmente, o uso do *Ktor Client* já estava estabelecido para a comunicação HTTP no *frontend* (alvo Android). Consequentemente, surgiu o dilema arquitetural para o *backend*: adotar o *Spring Boot*, que é um padrão tradicional da indústria.

Um dos fatores mais decisivo, validado pelo próprio *wizard* do KMP, foi a sinergia entre o cliente e o servidor. Utilizar *Ktor* em ambas as pontas permite uma partilha real de código (através de módulos *shared*). Foi possível partilhar *Data Transfer Objects* (DTOs), rotas, lógica de validação e configurações de serialização (JSON), eliminando a duplicação de código e reduzindo drasticamente a probabilidade de erros de integração entre a API e a aplicação móvel.

Outro fato é que o *Spring Boot* é um *framework* "pesado" e altamente opinativo, que traz consigo um ecossistema vasto de dependências (Injeção de Dependências, anotações complexas, reflexão). O *Ktor* segue uma filosofia *micro-framework* e minimalista baseada em *plugins*. O servidor arranca apenas com o estritamente necessário, resultando num tempo de inicialização muito mais rápido e menor consumo de memória.

Tendo em conta a decisão sobre a tecnologia da camada de acesso a dados, a combinação de *Ktor* com *Exposed* é o padrão recomendado pela JetBrains, permitindo um controlo mais direto e otimizado sobre as *queries* à base de dados usando código estritamente tipificado.

Capítulo 4

Arquitetura

4.1 Frontend

4.1.1 Biblioteca KMP para Mapa

Uma decisão feita foi criar uma biblioteca KMP nossa, utilizando *expect/actual* e as bibliotecas *MapBox Android* e *MapBox GL JS*.

Tomamos esta decisão, pois tendo em conta que já teríamos de implementar o *expect/actual* para estas 2 bibliotecas, achamos que criar uma biblioteca KMP e publica-la no *Maven Central*, seria uma forma de demonstrar a nossa contribuição para a comunidade KMP.

Esta biblioteca visa implementar mapa para Android e Web, da forma mais abstrata possível, mas deparamos-nos com alguns problemas.

Como o *MapBox GL JS* não está feito para *WASM*, tivemos de implementar em *jsMain*, mas ao decorrer do desenvolvimento da biblioteca deparamos-nos com outros problemas. Começamos por tentar utilizar a API *ComposeUI* para *JavaScript*, mas ainda é experimental e está inacabada por parte da JetBrains. Isso fez com que quando tentamos incluir *overlays* com o *ComposeUI* utilizado no Android não conseguíssemos visualiza-los.

Este problema ocorre porque o *ComposeUI* constroi um Canvas representado como um único elemento, que é um *WebGL* e o *MapBox GL JS* constroi o seu próprio *WebGL*. Acabou por ocorrer uma concorrência pela recursos de GPU onde não seria possível interligar os 2 *WebGL*. Por isso, tivemos de modificar para *ComposeHTML*, não podendo utilizar a abstração feita para o *ComposeUI* utilizado no *Android*.

Neste momento estamos a tentar abstrair com *expect/actual* as definições de *ComposeUI* e *ComposeHTML* para podermos ter o mesmo estilo de abstração nos 2 mapas. Apesar de ainda não estar implementado, estamos a avaliar utilizar a classe “Either”, falada na secção 3.2.2, para poder ter os valores separados.

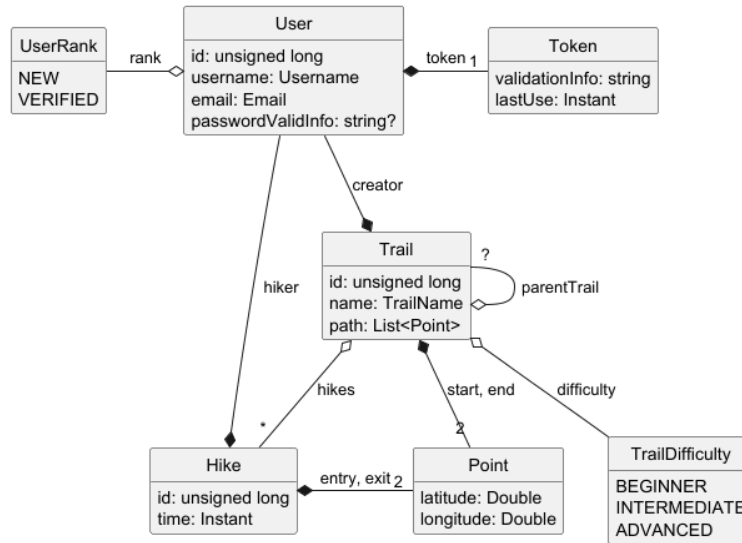


Figura 4.1: UML do Modelo de Dados desenvolvido

4.2 Backend

4.2.1 Modelo de Dados

O modelo de dados, figura 4.1, ilustra a arquitetura central do sistema de gestão de trilhos e registo de caminhadas projeto. O modelo assenta em 3 classes principais: User (o interveniente), Trail (o percurso) e Hike (a atividade).

User

A classe User, representação do cliente que utilizará a aplicação, contém alguns atributos de identificação como “id”, “username”, “email” e “passwordValidInfo”. É importante notar que passwordValidInfo pode ser nulo, isto porque projetamos 2 métodos de um utilizador se identificar e autenticar, por “email” e “password”, ou por *OAuth* da *Google* e neste caso, não há necessidade do utilizador ter uma password, mas obviamente, caso o utilizador se autentique por *OAuth*, não poderá utilizar o mesmo email para o método “email” e “password”.

O utilizador pode possuir 1 único token de sessão, apesar de haver possibilidade de alterar este valor apenas por uma variável. Como é uma relação de composição (losango preenchido), isto significa que o ciclo de vida do Token depende do User, logo se o utilizador for apagado do sistema, os seus tokens de validação perdem sentido e são destruídos.

O utilizador pode ser “creator” para Trails e “hiker” para Hikes. Esta relação de composição “creator” serve para sabermos quem foi o criador do percurso e termos uma ligação para que seja possível apagar trilhos. A relação de composição “hiker” serve para compreendermos quando um utilizador está ativamente a percorrer um trilho. Tal como o token, caso o utilizador seja apagado, os Hikes e Trails associados ao mesmo também serão.

O utilizador possui um `UserRank` que o categoriza (“NEW” ou “VERIFIED”). Esta categorização serve para habilitar o utilizador a poder criar trilhos e a evolução acontece quando o utilizador percorrer 10 caminhos ou 50 quilómetros, tendo como objetivo filtrar e reduzir a possibilidade de utilizadores com más intenções, já que com os recursos disponíveis, não existe possibilidade de fazer validação de percursos.

Trail

A classe `Trail` é a entidade central do domínio geográfico. Representa um percurso predefinido, identificado por um “id”, um “name” e constituído por um caminho (“path” da tipologia `List<Point>`).

O trilho está classificado pela sua dificuldade (`TrailDifficulty`: “Beginner”, “Intermediate”, “Advanced”), categorização alcançada a partir do tamanho do trilho.

Um trilho pode ser “parentTrail”, que indica a existência de uma ligação entre trilhos, pois caso o utilizador não queira percorrer o trilho completo, pode criar um sub-trilho para percorrer. Caso um trilho tenha um “parentTrail” que seja apagado, ele vira um trilho simples.

Um trilho tem 2 pontos principais, de classe `Point`, “start” e “end” que indicam os extremos do percurso, tendo também uma lista de `Point` que indica todos os pontos necessário o utilizador percorrer para determinar se completou ou não o percurso.

Um trilho “contém” várias caminhadas (* - zero ou mais). Se um trilho deixar de existir, os registos de atividades (`Hike`) realizadas exclusivamente nesse trilho perdem a sua referência principal, mas continuam a existir no sistema.

Hike

O `Hike` é o que podemos chamar de uma entidade de evento, que representa a ação temporal de um utilizador a percorrer um trilho. Tem um “id” e a marcação temporal (“time” do tipo `Instant`).

Uma caminhada, tal como o trilho, também recebe 2 pontos, “entry” e “exit”, já que o objetivo é um utilizador poder começar em qualquer extremo do percurso, sem ser preciso haver 2 trilhos, para o mesmo percurso, em direções diferentes.

4.2.2 Modelo ER

O modelo ER apresentado, figura 4.2 ilustra a adaptação física do modelo de domínio para a arquitetura de base de dados relacional. A essência das três entidades centrais (`User`, `Trail`, `Hike`) mantém-se, mas a estrutura foca-se agora em chaves primárias, determinadas pelos parâmetros a negrito, chaves estrangeiras (≪FK≫) e na simplificação de dados complexos.

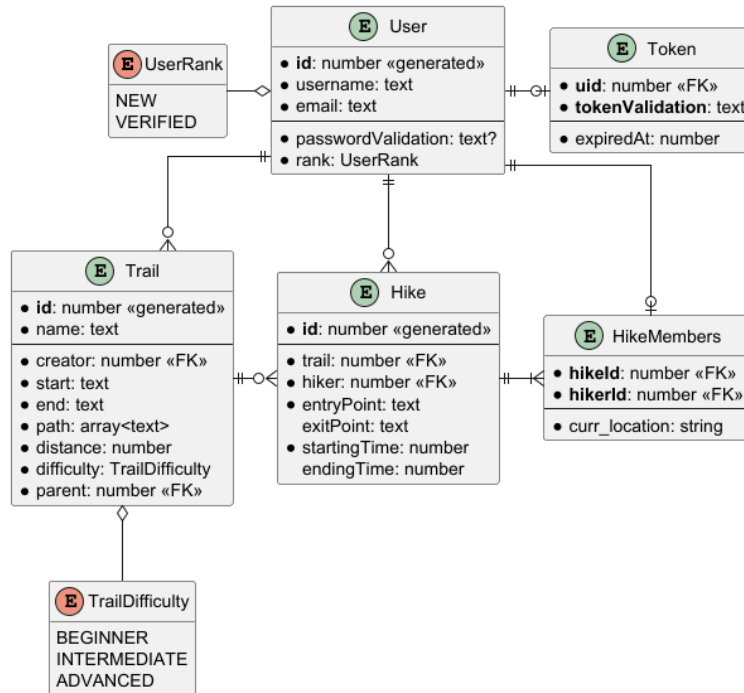


Figura 4.2: UML do Modelo ER desenvolvido

User

A lógica de negócio detalhada na secção 3.1.1 (autenticação mista com “passwordValidation” opcional) é visível no esquema físico com o sufixo “?”. A restrição de um único token de sessão por utilizador é garantida pela cardinalidade (1 para 0..1) entre a tabela User e Token, onde o Token utiliza o “id” do utilizador (uid) como chave estrangeira e em conjunto com o valor guardado na base de dados, fazem a sua chave primária.

Trail e Hike

Para otimizar o desempenho da base de dados, os dados geográficos e temporais foram simplificados para formatos nativos: Em Trail, os pontos “start” e “end” são guardados como texto (coordenadas serializadas), e o percurso completo (“path”) é armazenado num array de texto. A ligação obrigatória ao criador do trilho é assegurada pela chave estrangeira “creator”, “id” do utilizador. Em Hike, a mesma lógica aplica-se aos pontos de “entryPoint” e “exitPoint”. O atributo de tempo foi desdobrado em startingTime e endingTime (formato numérico, como timestamps Unix) para facilitar o cálculo da duração da atividade.

HikeMembers

A evolução mais significativa neste modelo ER face ao domínio inicial é a adição da tabela associativa HikeMembers. Enquanto a tabela Hike regista o evento e o seu criador/líder (através da chave estrangeira “hiker”), a tabela HikeMembers resolve a necessidade de termos

múltiplos utilizadores na mesma atividade. Ao cruzar o “id” da caminhada (“hikeId”) com os “id”s dos utilizadores envolvidos (“hikerId”), o sistema passa a suportar caminhadas em grupo. Adicionalmente, esta tabela inclui o atributo `curr_location`, permitindo monitorizar e persistir a localização individualizada de cada membro do grupo ao longo do percurso.

4.2.3 Rotas

A API criada para a finalidade deste projeto disponibiliza rotas destinadas à gestão de utilizadores, trilhos e caminhadas. Também é destinada a rota “/docs”, que apresenta uma página web que documenta as rotas descritas neste documento de forma interativa.

Salvo indicado em contrário, todas as rotas são acessíveis através do método HTTP GET, e formatam as suas respostas de acordo com a especificação JSON. Os tipos de retorno de cada rota apresentada estão associados ao seu sucesso; em caso de erro, qualquer rota responderá com uma mensagem de erro na sua resposta.

É relevante salientar que, nestas rotas, a notação “{x}” indica uma variável de caminho (neste exemplo, denominada “x”), necessária para o correto funcionamento da rota.

Utilizadores

As rotas apresentadas nesta secção possibilitam a gestão de utilizadores, permitindo ações como criação, autenticação ou eliminação.

- **Rotas sem autenticação**

- “/users/create”

- * Possibilita a criação de um utilizador
 - * **Corpo:** nome, email e palavra-passe; formato JSON
 - * **Retorno:** tokens de acesso e atualização e o seu instante de expiração; formato JSON

- “/users/login”

- * Possibilita a autenticação de um utilizador
 - * **Corpo:** email e palavra-passe; formato JSON
 - * **Retorno:** token e o seu instante de expiração; formato JSON

- **Rotas com autenticação por JWT**

- “/users/self”

- * Fornece os detalhes do próprio utilizador
 - * **Corpo:** nenhum
 - * **Retorno:** nome de utilizador e “rank”; formato JWT

- “/users/username”
 - * Possibilita a busca dos detalhes de um utilizador pelo seu nome
 - * **Corpo:** nenhum
 - * **Retorno:** nome de utilizador e “rank”; formato JWT
- “/users/{uid}/trails”
 - * Apresenta os trilhos criados pelo utilizador especificado
 - * **Corpo:** nenhum
 - * **Retorno:** lista de trilhos
- “/users/{uid}/stats”
 - * Apresenta as estatísticas pessoais do utilizador
 - * **Corpo:** nenhum
 - * **Retorno:** identificador do utilizador, número de trilhos percorridos, total de quilómetros percorridos, e total de tempo gasto em caminhadas
- Rotas com autenticação por *Bearer token*
 - “/users/refresh”
 - * Atualiza o token de acesso do utilizador, utilizado em rotas protegidas por JWT
 - * **Corpo:** nenhum
 - * **Retorno:** tokens de acesso e atualização e o seu instante de expiração; formato JSON
 - “/users/logout”
 - * Invalida os tokens gerados para o utilizador
 - * **Corpo:** nenhum
 - * **Retorno:** nenhum
 - “/users/delete”
 - * Elimina todos os dados pessoais do utilizador
 - * **Corpo:** nenhum
 - * **Retorno:** nenhum

Trilhos

As rotas aqui apresentadas possibilitam a consulta e gestão de trilhos, permitindo ações como criação, importação ou eliminação. Para estas rotas, foi aplicada a autenticação por JWT.

- “/trails/create”

- Possibilita a criação de um trilho.
- **Corpo:** nome, pontos inicial e final, lista de pontos intermédios, dificuldade e identificador do trilho parente; formato JSON
- **Retorno:** identificador do trilho
- “/trails/import”
 - Possibilita a importação de um trilho em formato KML
 - **Corpo:** ficheiro KML contendo pontos inicial e final, e lista de pontos intermédios; formato KML/XML, suportado por *MIME types* “application/xml”, “text/xml” ou “application/vnd.google-earth.kml+xml”
 - **Retorno:** identificador do trilho
- “/trails/available”
 - Apresenta uma lista paginada de trilhos disponíveis, sendo possível navegar entre páginas através dos parâmetros numéricos de pesquisa “skip” e “limit”
 - **Corpo:** nenhum
 - **Retorno:** lista paginada de trilhos
- “/trails/{tid}” (método GET)
 - Apresenta os detalhes de um trilho pelo seu identificador
 - **Corpo:** nenhum
 - **Retorno:** nome, pontos inicial e final, lista de pontos intermédios, dificuldade e identificador do trilho parente
- “/trails/{tid}” (método PUT)
 - Permite a atualização dos detalhes de um trilho pelo seu identificador
 - **Corpo:** nome novo do trilho
 - **Retorno:** nenhum
- “/trails/{tid}” (método DELETE)
 - Elimina um trilho pelo seu identificador
 - **Corpo:** nenhum
 - **Retorno:** nenhum
- “/trails/{tid}/start” (protocolo *WebSockets*)

- Sinaliza o início de uma caminhada num trilho pelo seu identificador; como utiliza o protocolo *WebSockets*, reutiliza o mesmo canal de comunicação para realizar o acompanhamento da caminhada em tempo real pelo cliente e pelo servidor
- **Corpo:** localização atual do cliente, enviada periodicamente em tempo real
- **Retorno:** localização atual de todos os caminhantes a frequentar o trilho, incluindo o utilizador atual

Caminhadas

As rotas aqui apresentadas possibilitam a consulta, finalização e cancelamento de caminhadas. Para estas rotas, foi aplicada a autenticação por JWT.

- “/hikes/{hid} (método GET)
 - Apresenta os detalhes de uma caminhada pelo seu identificador
 - **Corpo:** nenhum
 - **Retorno:** identificadores de caminhada, caminhante e trilho, pontos de entrada e saída, e instantes de início e fim da caminhada
- “/hikes/{hid} (método PUT)
 - Sinaliza a finalização de uma caminhada pelo seu identificador
 - **Corpo:** nenhum
 - **Retorno:** nenhum
- “/hikes/{hid} (método DELETE)
 - Sinaliza o cancelamento de uma caminhada pelo seu identificador
 - **Corpo:** nenhum
 - **Retorno:** nenhum

Capítulo 5

Conclusões

Tendo em conta os diversos problemas e desafios encontrados ao longo do desenvolvimento desta aplicação e solução proposta, a versão beta tem menos funcionalidades do que se tinha previsto inicialmente.

Contudo, prevê-se que, depois da ultrapassagem destes obstáculos, se consiga realizar o resto da aplicação mais tranquilamente e eficientemente, pelo que a versão final deste projeto terá quase ou todas as funcionalidades previstas na proposta entregue anteriormente.